SIMATS ENGINEERING

ASSIGNMENT 1 (CO1)

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Name: Rishitha

Register Number: 192425011

Nature of Assignment: Individual

Total Marks: 50

Compiler: g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), C++17

Compile command: g++ -std=c++17 -Wall -Wextra

GitHub Repository Link: https://github.com/_____/DSA01-192425011-Assignment1

PROBLEM SUMMARY AND ASSUMPTIONS

The Namma Metro network has three zones: Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge in peak hours (8 to 10 AM and 5 to 8 PM). A journey is allowed only if at least Rs 50 remains on the stored-value card after the fare is deducted. Students and senior citizens receive a concession. The first version of the gate software had fares off by a paisa, an inconsistent peak check, a negative balance and fare values typed in several places. The objective is to analyse, implement and validate the fare logic so that every fare and balance is exact, from one station (20,000 journeys a day) to the whole network (3,000,000 journeys a day).

The following assumptions were made where the scenario does not give a value:

- Peak hours are 08:00 to 09:59 and 17:00 to 19:59. The start time is included and the end time is excluded, so 10:00 and 20:00 are off-peak. This removes the inconsistency reported in the first version.
- Concession: students 25% and senior citizens 40%, applied after the peak surcharge.
- All money is stored as integer paise in long long. Percentages are calculated with integer arithmetic and rounded half-up to the nearest paisa: (paise x percent + 50) / 100.
- The repository contains one folder per task (Task01 to Task12). Every program was compiled with g++ -std=c++17 -Wall -Wextra and executed; the outputs and screenshots are in each folder's output directory.

TASK 1: UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

1.1 Objects, their data and behaviour

- **SmartCard:** holds the balance in paise. Behaviour: canTravel(fare), deduct(fare), recharge(amount), balance(). It is the only object allowed to change the balance, so it is responsible for the rule that at least Rs 50 must remain. This justifies making the balance private.
- **Passenger:** holds the name and category (General, Student, Senior). The category is a property of the person, not of the journey or the card, so it belongs here. Behaviour: category().
- **Journey:** holds the zone travelled, the hour and the minute of entry. Behaviour: fareFor(passenger). A journey is a separate object because the same card makes many journeys at different times.
- **Zone:** a value (Inner, Middle, Outer) with no behaviour of its own, so it is modelled as an enum class rather than a class. It is used as an index into the fare table.
- **FareTable:** holds the base fares, the surcharge and the concession percentages. Behaviour: isPeak(hour, minute) and fare(zone, time, category). Keeping all fare values in one object fixes the problem of "fare values typed in several places".

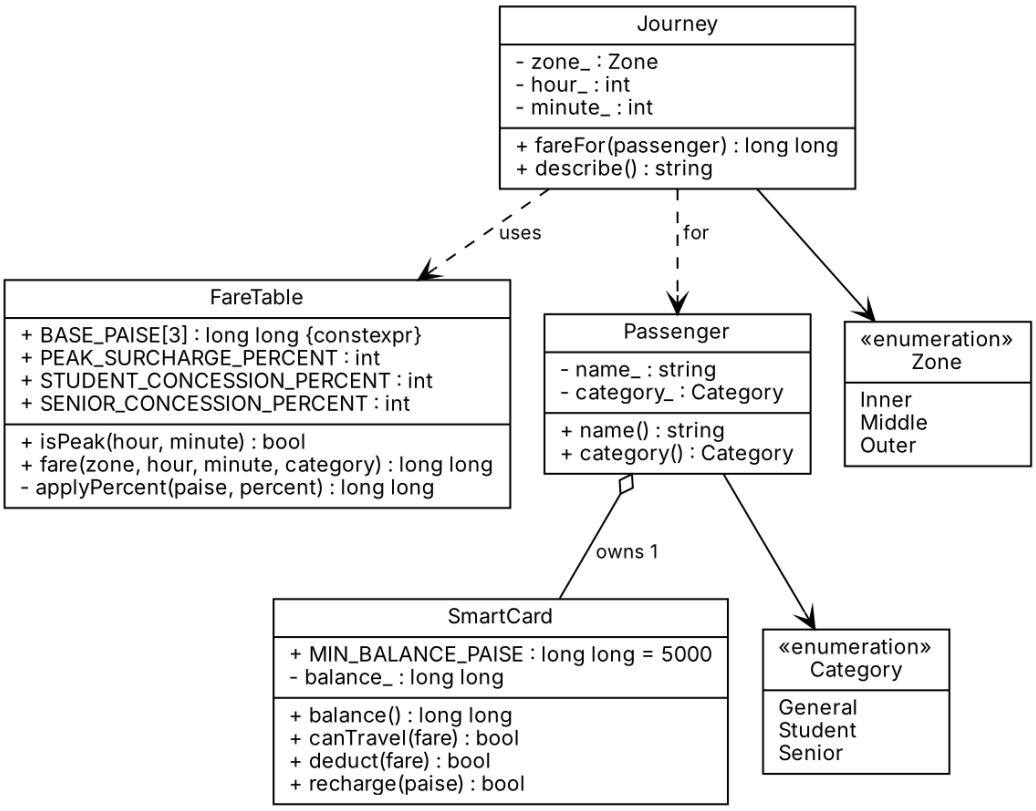


Figure 1: UML class diagram of the original design (Task01/task01_oop.cpp)

1.2 Procedural style versus object-oriented style

In a procedural style the program is a set of global variables (balance, fares, percentages) and free functions (fare(), travel()). Any function can read or change any variable, and the rules are

spread across the functions that happen to use them. In the object-oriented style the data is placed together with the functions that are allowed to change it. The program is organised around the objects of the problem (card, journey, fare table), not around the steps of the computation, and each class is responsible for keeping its own data valid.

1.3 The four principles of OOP in this system

- **Encapsulation:** SmartCard keeps balance_ private; it can only change through deduct() and recharge(), which enforce the Rs 50 rule and reject negative amounts.
- **Abstraction:** the gate code calls fare(journey) and card.deduct(fare) without knowing how the surcharge or concession is calculated.
- **Inheritance:** a future StudentCard or SeniorCard could derive from SmartCard; in Task 12 a MonthlyPass could share a common base class with other ticket types.
- **Polymorphism:** different fare rules (weekday, weekend, pass) could override a common virtual function such as computeFare(), so the gate treats all of them the same way.

1.4 Rules every correct journey must satisfy

- **R1:** balance after the journey is at least Rs 50; otherwise the journey is denied and the balance is unchanged. Justification: stated in the scenario, and protects against a negative balance.
- **R2:** the fare is the base fare of the farthest zone; the 20% surcharge applies only for start times in [08:00, 10:00) and [17:00, 20:00).
- **R3:** every fare and balance is an exact whole number of paise; no rounding error may accumulate.
- **R4:** fares, recharges and balances are never negative.

1.5 Alternative concept and complexity analysis

Task01/task01_procedural.cpp implements the same logic with global variables and free functions. Both versions take O(1) time per journey (a fixed number of comparisons and multiplications) and O(1) memory per card (one long long of 8 bytes). The difference is in maintenance: when one fare rule changes (for example, the minimum balance), the object-oriented version changes one constant inside SmartCard, and every journey automatically obeys it. In the procedural version every function that changes g_balance must be found and checked. The program output shows the risk: after one stray statement g_balance -= 20000 the balance became -126 rupees, which the private member in the class version makes impossible.

Evidence (Task01/output/task01_oop_output.txt):

```
$ g++ -std=c++17 -Wall -Wextra  -o task01_oop task01_oop.cpp
$ ./task01_oop
Passenger: Rishitha (Student)
Opening balance: Rs 150.00

Journey         Fare (Rs)   Result    Balance (Rs)
Inner 07:45     7.50        OK        142.50
Outer 08:00     27.00       OK        115.50
Middle 10:00    15.00       OK        100.50
```

```
Outer 19:59     27.00       OK        73.50

Negative recharge accepted? no
Final balance: Rs 73.50
[exit code 0]
```

TASK 2: USE TOKENS, KEYWORDS, IDENTIFIERS AND CONSTANTS CORRECTLY

2.1 The five kinds of tokens

For the statement long long fareNow = fare::BASE_PAISE[2] * 120 / 100; the tokens are:

- **Keywords:** long, long (also constexpr, enum, class, namespace, return, switch, case, const elsewhere).
- **Identifiers:** fareNow, fare, BASE_PAISE.
- **Constants (literals):** 2, 120, 100 (also 1000, 5000, 'A', "Fare Rs").
- **Operators:** =, ::, [ ], *, /.
- **Punctuators:** ; [ ] { } ( ) ,

2.2 #define, const and constexpr

- **#define** is a text replacement done by the preprocessor. It has no type, no scope and no name in the debugger, and missing brackets change the meaning of an expression.
- **const** creates a typed, scoped, read-only variable, but its value may be fixed only at run time.
- **constexpr** creates a typed, scoped constant whose value must be known at compile time. It can be used in array sizes, static_assert and other constexpr functions. This is why it is preferred for the fare table: the compiler checks the values, the type is exact (long long paise), and a revision is made in one place.

```
namespace fare {
    enum class Zone : int { Inner, Middle, Outer, Count };
    constexpr long long BASE_PAISE[static_cast<int>(Zone::Count)] = {1000, 2000, 3000};
    constexpr int PEAK_SURCHARGE_PERCENT = 20;
    constexpr long long MIN_BALANCE_PAISE = 5000;
}
static_assert(fare::peakFare(Zone::Outer) == 3600, "Outer peak fare must be Rs 36");
```

2.3 enum and enum class

A plain enum converts silently to int and its names leak into the surrounding scope. In the program, OUTER == BLUE compiled and evaluated to true, because both are the integer 2 (g++ only issued -Wenum-compare). An enum class is scoped (Zone::Outer), does not convert implicitly to int, and cannot be compared with another enumeration, so such mistakes become compile errors. It is therefore safer for zones.

2.4 Naming rules and conventions

- Rules: an identifier starts with a letter or underscore, contains only letters, digits and underscores, is case-sensitive and cannot be a keyword. Names beginning with an underscore followed by a capital letter, or containing a double underscore, are reserved.
- Conventions used: UPPER_SNAKE_CASE for constants (MIN_BALANCE_PAISE), PascalCase for types (SmartCard, Zone), camelCase for functions and variables

(canTravel, farePaise), a trailing underscore for private members (balance_), and the unit in the name (_PAISE, _PERCENT).

2.5 Alternative concept and complexity analysis

Task02/task02_macros.cpp writes the same values as #define macros. A macro has no type checking and no scope, so the error is found only at run time. #define PEAK_FACTOR 100 + 20 makes OUTER_FARE * PEAK_FACTOR / 100 expand to 3000 * 100 + 20 / 100 = 300000 paise instead of 3600. A macro occupies no storage; a constexpr scalar also usually occupies no storage because the compiler substitutes the value (the 3-element array occupies 24 bytes only if it is indexed at run time). The running time is the same in both cases because the values are constants in the machine code. The only difference is safety.

```
$ g++ -std=c++17 -Wall -Wextra  -o task02_macros task02_macros.cpp
$ ./task02_macros
Outer peak fare with macro  : 300000 paise (expected 3600)
SQUARE(1 + 1) with macro    : 3 (expected 4)
MIN_BALANCE has no name in the debugger, only the value 5000
Correct value with brackets : 3600 paise
[exit code 0]
```

TASK 3: CHOOSE DATA TYPES AND CHECK TYPE COMPATIBILITY

3.1 Data types chosen (sizes measured on this system)

- **Fare, balance, recharge, totals:** long long paise, 8 bytes, range up to 9,223,372,036,854,775,807. Exact and large enough for any total.
- **Hour, minute, percentages, zone number typed by the operator:** int, 4 bytes.
- **Zone:** enum class Zone (underlying int, 4 bytes).
- **Peak flag, allowed flag:** bool, 1 byte.
- **Category letter typed at the gate:** char, 1 byte (G, S or C).
- **Journey count:** long long. 3,000,000 fits in int, but it is multiplied with fares, so the wider type avoids overflow in expressions.
- **auto:** used only where the type is obvious; auto journeys = 3000000 deduces int (4 bytes).
- **float and double:** not used for money; double is used only for display (average fare) and timing.

3.2 Why money is stored in integer paise

Binary floating point cannot represent most decimal fractions exactly. The program shows that 0.1 + 0.2 == 0.3 is false and that 10 x 1.2 x 0.6 is 7.1999999999999993; converting it to paise by truncation gives 719 instead of 720, exactly the "off by a paisa" fault in the first version. Integer paise are exact: (1000 x 120 / 100) x 60 / 100 = 720.

3.3 Type compatibility in mixed expressions

- In int and double expressions, the int is converted to double (usual arithmetic conversions) and the result is double.
- bool is promoted to int (false = 0, true = 1) in arithmetic: fare + peak * 200 is legal but unclear.
- char and short are promoted to int before arithmetic.
- When int and unsigned int of the same size meet, the int is converted to unsigned, so -1 becomes 4,294,967,295 (Task 4).
- Assigning a double to an int truncates toward zero; assigning a long long to an int may lose the high bits.

3.4 Ranges and the network total

INT_MAX is 2,147,483,647 paise, about Rs 2.15 crore. The worst case for one day across the network is 3,000,000 x 3600 = 10,800,000,000 paise, five times INT_MAX, so int is not enough. With the realistic journey mix of Task 10 the int total overflowed at journey 1,095,975. long long holds the total with a large margin.

3.5 Alternative concept and complexity analysis

If money were stored as double rupees, each addition is still O(1) and each value still occupies 8 bytes, the same as long long. The difference is accuracy. Adding Rs 21.60 n times in double gave the following measured errors, which grow with n, while the paise total stays exact:

```
=== Rounding error growth: adding n fares of Rs 21.60 ===
n         double total (Rs)         exact paise total     error (Rs)
10        216.0000000000            21600                 -2.842e-14
1000      21599.9999999998          2160000               -1.564e-10
20000     431999.9999998668         43200000              -1.332e-07
500000    10799999.9998988397       1080000000            -1.012e-04
3000000   64800000.0031086951       6480000000            3.109e-03

float total for 3,000,000 x 21.60 = 62883672.00 (exact 64800000.00)
[exit code 0]
```

Resolution: store paise in long long, use integer percentages with explicit rounding, and convert to rupees only for printing.

TASK 4: CONTROL IMPLICIT CONVERSIONS

4.1 Conversion points in the fare program

- **Integer division and integer-to-double:** base * 20 / 100 is integer division; in double c = a / b the division happens in int first and only the result 3 is converted to 3.0.
- **Signed and unsigned comparison:** comparing an int index with vector::size() or an unsigned counter converts the int to unsigned; g++ warns -Wsign-compare.
- **Narrowing:** int x = 3.7 silently stores 3; long long to int may lose data. Brace initialisation int x{3.7}; is an error, which prevents the narrowing.
- **static_cast:** needed when converting enum class to int for array indexing, when a deliberate real division is needed (static_cast<double>(a) / b), and when printing a char as a number.

4.2 Predict the Output

My predicted output (written before running):

```
3 3.5
C 68
1 true
s >= u
```

Actual output (Task04/output/task04_predict_output.txt):

```
$ g++ -std=c++17 -Wall -Wextra  -o task04_predict task04_predict.cpp
task04_predict.cpp: In function 'int main()':
task04_predict.cpp:20:21: warning: comparison of integer expressions of different signedness: 'int' and
'unsigned int' [-Wsign-compare]
   20 |     std::cout << (s < u ? "s < u" : "s >= u") << '\n';
      |                   ~~^~~
$ ./task04_predict
3 3.5
C 68
1 true
s >= u
[exit code 0]
```

The prediction matched the actual output. Explanation of each line:

- **Line 1 "3 3.5":** a / b with two ints is integer division (7 / 2 = 3); the int 3 is then converted to double 3.0 and printed as 3. In d, a is converted first, so the division is 7.0 / 2.0 = 3.5.
- **Line 2 "C 68":** 'A' + 2 promotes the char to int (65 + 2 = 67), which is converted back to char C. In ch + 1 the char is promoted to int, so the int 68 is printed, not the letter D.
- **Line 3 "1 true":** int to bool: 5 is non-zero, so flag is true. Without boolalpha a bool prints as 1; with boolalpha it prints as true.
- **Line 4 "s >= u":** signed to unsigned: s is converted to unsigned, -1 becomes 4,294,967,295, which is greater than 1. The compiler warned: comparison of integer expressions of different signedness [-Wsign-compare].

4.3 Alternative concept and complexity analysis

Task04/task04_explicit.cpp writes every conversion with static_cast and uses brace initialisation everywhere. The output becomes 3 3.5 / C D / 1 true / s < u, because each intention is now stated. Running time and memory do not change: a static_cast produces the same machine instruction as the implicit conversion, and no extra storage is used. The benefit is that errors move from run time to compile time: Task04/task04_narrowing_error.cpp fails to compile with "error: narrowing conversion of 3.7 from double to int [-Wnarrowing]", and the narrowing long long to int is diagnosed by -Wnarrowing.

TASK 5: WRITE CORRECT EXPRESSIONS WITH THE RIGHT PRECEDENCE

5.1 Fare expressions

```
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
fare += peak ? fare * PEAK_SURCHARGE_PERCENT / 100 : 0;
fare -= student ? fare * STUDENT_CONCESSION_PERCENT / 100 : 0;
```

- Relational operators (>=, <) are evaluated before && and &&, before ||, so each bracket is one time window. || and && evaluate left to right and short-circuit.
- In the surcharge, * and / have equal precedence and associate left to right: (fare * 20) / 100, so multiplication happens before the integer division and no paise are lost.
- The conditional operator ?: has lower precedence than arithmetic, and the compound assignment += has the lowest precedence and associates right to left, so the whole ?: expression is evaluated first and then added to fare.
- **x++ and ++x:** x++ yields the old value and then increments; ++x increments and yields the new value. i = i++ is a mistake: before C++17 it is undefined behaviour, and in C++17 the increment is simply lost (i stays 5). g++ warned "operation on i may be undefined [-Wsequence-point]".
- **/ and % with negative operands:** since C++11 division truncates toward zero, and the remainder has the sign of the dividend: -7 / 2 = -3, -7 % 2 = -1, 7 % -2 = 1.
- Because && binds tighter than ||, hour >= 8 && hour < 10 || hour >= 17 && hour < 20 gives the same result for all 24 hours (verified by the program), but the reader must remember the precedence table, and g++ warns -Wparentheses. The bracketed form shows the intention directly.

5.2 Predict the Output

My predicted output (written before running):

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Actual output (Task05/output/task05_predict_output.txt):

```
$ g++ -std=c++17 -Wall -Wextra  -o task05_predict task05_predict.cpp
$ ./task05_predict
11 16
9 6 4
6 4
3 1 -3 -1
18
[exit code 0]
```

The prediction matched the actual output.

- **Line 1:** * has higher precedence than +: 5 + (3 x 2) = 11. The brackets force (5 + 3) first: 8 x 2 = 16.
- **Line 2:** x++ yields 5 and then x becomes 6; ++y makes y 4 and yields 4; + gives 9, assigned with =. Output 9 6 4.
- **Line 3:** > is evaluated first (6 > 4 is true), then ?: selects x = 6. % and * have equal precedence and associate left to right: (6 % 4) x 2 = 2 x 2 = 4.
- **Line 4:** 10 / 3 = 3 and 10 % 3 = 1. Unary minus binds first, so -10 / 3 = -3 (truncation toward zero) and -10 % 3 = -1 (sign of the dividend).
- **Line 5:** the loop condition i < 10 is checked before each pass and i += 3 runs after each pass: i = 0, 3, 6, 9, so k = 18.

5.3 Alternative concept and complexity analysis

With a lookup table bool peak[24] indexed by the hour, the check becomes one array access instead of up to four comparisons and two logical operators. Both are O(1). The table costs 24 bytes of extra space. Measured with -O2 for 100,000,000 checks: expression 79.6 ms, table 62.0 ms. The table cannot represent minutes (for example 09:59) unless it grows to 1440 entries, so the expression on minutes of the day is kept in the final design and the table is used only to verify it.

TASK 6: DESIGN THE CONTROL STRUCTURES

6.1 Choice of control structures

- **switch on the zone:** used where one value selects one of several fixed cases. With an enum class and no default, g++ warns -Wswitch if a new zone is not handled.
- **if-else:** used for range conditions such as the peak window and the minimum balance, which a switch cannot test.
- **conditional operator ?: :** used for short value choices, for example adding the surcharge or 0.
- **Loops:** a range-based for loop is best for a fixed array of journeys (no index errors); a counted for loop when the index is needed; a while loop when reading journeys until the input ends.
- **continue** skips one invalid journey (unknown zone, hour 25) and processes the next; **break** stops the loop when nothing more can be processed.
- **When no case matches:** control passes to the end of the switch. In the completed code base stays 0, which would be a free journey; this is why the -Wswitch warning and validation of the zone number before the cast are important.

6.2 Pseudocode (SIMATS Engineering Standard for Algorithmic Pseudocode)

```
ALGORITHM ComputeFare
INPUT : zone (Inner | Middle | Outer), peak (Boolean)
OUTPUT: fare in paise
BEGIN
    SET base <- 0
    CASE zone OF
        Inner  : SET base <- BASE_PAISE[0]
        Middle : SET base <- BASE_PAISE[1]
        Outer  : SET base <- BASE_PAISE[2]
    END CASE
    IF peak = TRUE THEN
        SET base <- base + (base * PEAK_SURCHARGE_PERCENT) DIV 100
    END IF
    RETURN base
END

ALGORITHM CanTravel
INPUT : balance, fare (paise)
OUTPUT: TRUE if the journey is allowed
BEGIN
    IF balance - fare >= MIN_BALANCE_PAISE THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END

ALGORITHM ProcessDay
INPUT : list of trips, opening balance
BEGIN
    FOR EACH trip IN trips DO
        IF trip is invalid THEN CONTINUE END IF
```

```
        SET f <- ComputeFare(trip.zone, IsPeak(trip.hour,
trip.minute))
        IF CanTravel(balance, f) THEN
            SET balance <- balance - f
            PRINT "Fare", f, "balance", balance
        ELSE
            PRINT "Denied (low balance)"
        END IF
    END FOR
END
```

6.3 Complete the Code

```
long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak)
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;
}
```

Actual output (matches the expected output exactly):

```
$ g++ -std=c++17 -Wall -Wextra  -o task06_complete
task06_complete.cpp
$ ./task06_complete
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
[exit code 0]
```

Journeys at exactly 10 AM and 8 PM (Task06/output/task06_peak_boundary_output.txt):

```
Zone   Time   Peak  Fare(Rs)  Balance(Rs)
Inner  07:59  no    10.00     490.00
Inner  08:00  yes   12.00     478.00
Middle 09:59  yes   24.00     454.00
Middle 10:00  no    20.00     434.00
Outer  16:59  no    30.00     404.00
Outer  17:00  yes   36.00     368.00
Outer  19:59  yes   36.00     332.00
Outer  20:00  no    30.00     302.00
skip   09:00  invalid journey ignored
skip   25:00  invalid journey ignored
10:00 peak? false   20:00 peak? false   (both correctly off-peak)
[exit code 0]
```

6.4 Alternative concept and complexity analysis

With constexpr long long FARE[] = {1000, 2000, 3000} the base fare is FARE[static_cast<int>(z)]. Both lookups are O(1); a dense switch is usually compiled into a jump table or, at -O2, into a table lookup as well. Measured for 100,000,000 lookups at -O2: switch 78.6 ms, array 69.5 ms. The array occupies 24 bytes. For a fourth zone, the array needs one new value (and a static_assert can check its size against the enum), whereas the switch needs a new case in every switch on the zone. Missing it gives fare 0, as shown in Task 12. The array therefore copes better with change.

TASK 7: FROM PROCEDURAL TO OBJECT-ORIENTED

7.1 Procedural version

Task07/task07_procedural.cpp keeps the balance in a separate variable and provides free functions canTravel(balance, fare) and deduct(balance, fare). Correctness depends on every caller calling canTravel() before deduct(). The program shows a careless caller skipping the check, giving a balance of -3600 paise.

7.2 Object-oriented version

```
class SmartCard {
public:
    static constexpr long long MIN_BALANCE_PAISE = 5000;
    explicit SmartCard(long long openingPaise);
    bool canTravel(long long fare) const { return fare >= 0 && balance_ - fare >= MIN_BALANCE_PAISE; }
    bool deduct(long long fare) { if (!canTravel(fare)) return false; balance_ -= fare; return true; }
    bool recharge(long long paise) { if (paise <= 0) return false; balance_ += paise; return true; }
    long long balance() const { return balance_; }
private:
    long long balance_;
};
```

7.3 How the class prevents a negative balance

balance_ is private, so the only ways to change it are deduct() and recharge(). deduct() always calls canTravel() first and refuses negative fares; recharge() refuses zero or negative amounts; the constructor clamps a negative opening value to 0. Since every path keeps balance_ at or above Rs 50 after a journey, the invariant cannot be broken from outside the class. An attempt such as card.balance_ = -100 is a compile error.

Output (Task07/output/task07_oop_output.txt):

```
paid 3600, balance 6400
denied 2000, balance 6400
paid 1000, balance 5400
deduct(9000) allowed? no, balance 5400
deduct(-500) allowed? no
recharge(-2000) allowed? no
Balance can never go below 5000 paise through this class.
[exit code 0]
```

7.4 Which version is easier to change

When the minimum balance changes, the object-oriented version changes one constant, MIN_BALANCE_PAISE, inside SmartCard, and all uses follow. In the procedural version the constant may also be in one place, but every function that changes the balance must still be reviewed to make sure it checks the rule. The object-oriented version is therefore easier and safer to change.

7.5 Alternative concept and complexity analysis

If the balance is a public data member (Task07/task07_public_balance.cpp), time and space do not change: both cards are 8 bytes, and inline member functions are as fast as direct access. However, any part of the program can write card.balance -= 9000, which produced a balance of -2600 paise. The rule "balance never below Rs 50 after a journey" (and even "balance never negative") can then be broken anywhere, and finding the faulty line requires searching the whole program.

TASK 8: READ AND PRINT DATA SAFELY

8.1 Reading with cin and checking cin.fail()

When the operator types letters where a number is expected, cin >> v fails: v is not set, the fail bit is set, and every later read fails as well. The program checks the stream after every read (if (std::cin >> v)), and also checks the range (zone 1 to 3, hour 0 to 23, minute 0 to 59, recharge 1 to 10000). End of input is detected with cin.eof() so the program cannot loop forever.

8.2 Clearing the error state

```
std::cin.clear();                                   // 1. reset the fail state
std::string bad; std::cin >> bad;                   // 2. read the bad word (to report it)
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');   // 3. discard the rest of the line
```

8.3 Formatting money and the journey report

Money is printed from integer paise: rupees = paise / 100, and the paise part with std::setw(2) and std::setfill('0') so that 5 paise prints as .05. Columns are aligned with std::left / std::right and std::setw. std::fixed and std::setprecision(2) are used only for the derived average fare.

Run with invalid input (Task08/output/task08_safe_io_invalid_output.txt):

```
=== Namma Metro Gate ===
Recharge amount in Rs: abc  -> not a number, try again
Recharge amount in Rs: -100  -> out of range (1 to 10000), try again
Recharge amount in Rs: 200
Number of journeys: 3
-- Journey 1 --
Zone (1 Inner, 2 Middle, 3 Outer): x  -> not a number, try again
Zone (1 Inner, 2 Middle, 3 Outer): 7  -> out of range (1 to 3), try
again
Zone (1 Inner, 2 Middle, 3 Outer): 3
Hour (0-23): ten  -> not a number, try again
Hour (0-23): 8
Minute (0-59): 0
Category (G/S/C): S
-- Journey 2 --
Zone (1 Inner, 2 Middle, 3 Outer): 2
Hour (0-23): 9
Minute (0-59): 75  -> out of range (0 to 59), try again
Minute (0-59): 30
Category (G/S/C): Q  -> use G (general), S (student) or C (senior
citizen)
Category (G/S/C): C
-- Journey 3 --
Zone (1 Inner, 2 Middle, 3 Outer): 1
Hour (0-23): 20
Minute (0-59): 0
Category (G/S/C): G

=========================================================
                  JOURNEY REPORT
=========================================================
No  Zone    Time   Cat    Fare(Rs)   Status  Balance(Rs)
---------------------------------------------------------
```

```
1   Outer   08:00  S         27.00       OK       173.00
2   Middle  09:30  C         14.40       OK       158.60
3   Inner   20:00  G         10.00       OK       148.60
-------------------------------------------------------
Total paid: Rs 51.40   Average fare: Rs 17.13
[exit code 0]
```

8.4 Alternative concept and complexity analysis

Task08/task08_getline.cpp reads each line with std::getline() and converts it with std::stoi() inside try/catch. Both approaches take O(L) time for a line of length L. cin >> uses only the stream buffer, while getline needs an extra std::string of O(L) space for the line. Recovery is simpler with getline: a bad line is discarded as a whole, the stream never enters the fail state, and the reason is known (invalid_argument, out_of_range). It also detects input such as "12abc", which cin >> silently reads as 12, by checking that the whole line was used.

TASK 9: IMPLEMENT AND TEST THE FARE LOGIC

The complete fare logic is in Task09/metro_fare.hpp (constants, isPeak, fare, zoneFromNumber, parseHour, SmartCard). Task09/task09_tests.cpp contains 30 separate checks covering normal journeys, boundary cases, invalid cases and a five-journey set calculated by hand.

9.1 Hand calculation of five journeys

- Journey 1: Inner, 13:00, general: Rs 10.00
- Journey 2: Middle, 09:15 peak, general: 20 x 1.2 = Rs 24.00
- Journey 3: Outer, 11:00, student: 30 x 0.75 = Rs 22.50
- Journey 4: Outer, 18:00 peak, senior: 36 x 0.60 = Rs 21.60
- Journey 5: Middle, 21:00, senior: 20 x 0.60 = Rs 12.00

Total = Rs 90.10. Opening balance Rs 200.00, closing balance Rs 109.90.

9.2 Test table: expected and actual results

ID	Test case	Expected	Actual	Result
N1	Inner off-peak 12:00	10.00	10.00	PASS
N2	Inner peak 09:00	12.00	12.00	PASS
N3	Middle off-peak 14:30	20.00	20.00	PASS
N4	Middle peak 18:15	24.00	24.00	PASS
N5	Outer off-peak 22:00	30.00	30.00	PASS
N6	Outer peak 08:30	36.00	36.00	PASS
N7	Outer peak student (36 x 0.75)	27.00	27.00	PASS
N8	Outer peak senior (36 x 0.60)	21.60	21.60	PASS
B1	07:59 is off-peak	no	no	PASS
B2	08:00 is peak	yes	yes	PASS
B3	09:59 is peak	yes	yes	PASS
B4	10:00 is off-peak	no	no	PASS
B5	17:00 is peak	yes	yes	PASS
B6	19:59 is peak	yes	yes	PASS
B7	20:00 is off-peak	no	no	PASS
B8	Rs 80 - Rs 30 leaves exactly Rs 50: allowed?	yes	yes	PASS
B9	Balance after B8	50.00	50.00	PASS
B10	Rs 79.99 - Rs 30 leaves Rs 49.99: allowed?	no	no	PASS
B11	Balance unchanged after denial	79.99	79.99	PASS
I1	Unknown zone number 4 rejected	rejected	rejected	PASS
I2	Zone number 0 rejected	rejected	rejected	PASS
I3	Negative recharge -500 rejected	rejected	rejected	PASS
I4	Balance unchanged after I3	100.00	100.00	PASS
I5	Negative fare cannot be deducted	rejected	rejected	PASS
I6	Hour typed as "ab" rejected	rejected	rejected	PASS
I7	Hour typed as "9x" rejected	rejected	rejected	PASS
I8	Hour typed as "24" rejected	rejected	rejected	PASS
I9	Hour typed as "17" accepted	17	17	PASS
T1	Total of 5 journeys (hand: 90.10)	90.10	90.10	PASS
T2	Balance after 5 journeys (hand: 109.90)	109.90	109.90	PASS

Passed 30, failed 0

9.3 Alternative concept and complexity analysis

In Task09/task09_table_tests.cpp each case is one row of a constant table {id, zone, hour, minute, category, expected} and a single loop runs all rows. Running t tests takes O(t) time in both approaches. The table needs O(t) space: 17 cases x 32 bytes = 544 bytes (measured). A new boundary case is added by writing one line in the table, with no new code, which makes it easy to cover every boundary. Separate checks are better for special cases such as balance changes and invalid input that do not fit one row format, so both are used.

```
F17  expected 12.00   actual 12.00   PASS
17 table cases, 17 passed, 0 failed
Table size: 544 bytes (32 bytes per case)
```

TASK 10: COMPARE EXPECTATIONS WITH EXECUTION

Task10/task10_totals.cpp processes a fixed mix of journeys (zones in rotation, 40% in peak hours, one in seven students, one in seven senior citizens) and adds the fares in four ways.

```
========== n = 20000 journeys ==========
long long paise (exact) : Rs 391876.10
double rupees           : Rs 391876.100000   difference 2.736e-09
float rupees            : Rs 391875.12   difference -0.97
int paise               : 39187610   no overflow
journeys whose double fare truncates to a wrong paisa: 761

========== n = 500000 journeys ==========
long long paise (exact) : Rs 9797140.40
double rupees           : Rs 9797140.400000   difference 5.029e-08
float rupees            : Rs 9795510.00   difference -1630.40
int paise               : 979714040   no overflow
journeys whose double fare truncates to a wrong paisa: 19047

========== n = 3000000 journeys ==========
long long paise (exact) : Rs 58782863.40
double rupees           : Rs 58782863.400056   difference 5.594e-05
float rupees            : Rs 58338768.00   difference -444095.40
int paise               : 1583319044   OVERFLOWED at journey 1095975
journeys whose double fare truncates to a wrong paisa: 114284

INT_MAX = 2147483647 paise = Rs 21474836
50,000,000 additions: long long 26.794 ms, double 107.885 ms, int 28.157 ms
sizes: int 4, long long 8, double 8, float 4 bytes
```

10.1 Double and integer-paise totals

Expected: the double totals would differ from the paise totals. Observed: the double totals agree with the exact total to within Rs 0.000000003 (20,000 journeys), Rs 0.00000005 (500,000) and Rs 0.000056 (3,000,000). The difference grows with the number of journeys but is still below one paisa for the total. The larger problem is in individual fares: 114,284 of the 3,000,000 double fares (about 3.8%) become one paisa less when converted to paise by truncation, for example 7.20 becomes 7.19. float is much worse: Rs 0.97 wrong for 20,000 journeys and Rs 4,44,095.40 wrong for 3,000,000.

10.2 Does int overflow?

Yes. The exact network total is 5,878,286,340 paise, but INT_MAX is 2,147,483,647. The int accumulator overflowed at journey 1,095,975 and wrapped to 1,583,319,044, a value that looks plausible but is wrong. Signed overflow is undefined behaviour in C++, so the program detects it with __builtin_add_overflow and simulates the wrap with unsigned arithmetic.

10.3 Compiler warnings with -Wall -Wextra

- **-Wswitch:** "enumeration value Outer not handled in switch". The Outer fare would be 0.

- **-Wparentheses:** "suggest parentheses around && within ||". The expression is correct, but unclear.
- **-Wunused-parameter:** "unused parameter minute". The peak check ignores minutes, so 09:59 and 09:00 cannot be told apart.
- **-Wsign-compare:** "comparison of integer expressions of different signedness". int compared with vector::size_type.
- **-Wunused-variable:** "unused variable unusedTotal". Dead code, often a sign of a forgotten calculation.
- In the other tasks: -Wsign-compare (Task 4), -Wnarrowing (Task 4), -Wenum-compare (Task 2) and -Wsequence-point (Task 5). Note that int paise = rupees * 100 produced no warning; it needs -Wconversion.

10.4 Reasons for the differences

- double stores 52 bits of mantissa, so values like 21.6 and 7.2 are stored approximately; every addition rounds again, and the error accumulates with n.
- float has only 24 bits of mantissa (about 7 decimal digits); once the total exceeds about Rs 1,00,000, the paise digits are lost on every addition.
- int is 32 bits; the network total needs about 33 bits.

10.5 Alternative concept and complexity analysis

At 3,000,000 journeys an int fails by overflow (at journey 1,095,975 in this mix; in the worst case of all Rs 36 fares, at journey 596,524). A double does not overflow but loses precision (error 5.6 x 10^-5 rupees and 3.8% of single fares off by a paisa). A long long is exact and would overflow only after about 2.5 x 10^15 such journeys. Each addition is O(1) for every type. Measured for 50,000,000 additions: long long 26.8 ms, int 28.2 ms and double 107.9 ms (floating-point addition has a longer latency). Space per variable: int 4 bytes, float 4 bytes, long long 8 bytes, double 8 bytes. long long is therefore both the most accurate and among the fastest choices.

TASK 11: RECOMMEND AND DEFEND THE FINAL DESIGN

11.1 Recommended types, constants and classes

- **Money:** long long integer paise, because Task 3 showed double gives 719 instead of 720 paise and Task 10 showed 114,284 wrong single fares in double and an int overflow at journey 1,095,975.
- **Constants:** one constexpr set in a namespace and enum class Zone, because Task 2 showed a macro producing 300000 instead of 3600 paise, and a plain enum allowing OUTER == BLUE.
- **Fare table:** constexpr std::array indexed by the zone, with static_assert on its size, because Task 6 showed it is at least as fast as the switch and needs only one new value for a new zone.
- **Peak rule:** minutes of the day with the window [08:00, 10:00) and [17:00, 20:00), written with brackets (Task 5), verified at 07:59, 08:00, 09:59, 10:00, 16:59, 17:00, 19:59, 20:00 (Tasks 6 and 9).
- **Classes:** FareCalculator for the rules, SmartCard with a private balance (Task 7 showed both public and procedural versions allow a negative balance), Journey, MonthlyPass and Gate (Task 12).
- **Input:** line-based reading with validation and recovery (Task 8). **Testing:** a table of test cases run after every change (Task 9).

11.2 How a fare revision is carried out

- 1. Change the value in the single constexpr fare table (or in the configuration file, see 11.5).
- 2. Update the expected values in the test table and add a row for any new boundary.
- 3. Compile with -Wall -Wextra and confirm that there are no warnings and that static_assert checks pass.
- 4. Run the test table; deploy only if all tests pass.

11.3 Limitations

- The fare table is compiled into the program, so every revision needs a rebuild and redeployment to every gate.
- Concession percentages are assumptions; the order of rounding (surcharge, then concession) must be confirmed by the metro authority.
- Each card is handled by one gate at a time; concurrent updates from several gates, persistence and daily reconciliation are outside this program.
- The "farthest zone" is supplied by the caller; the station-to-zone mapping is not modelled.

11.4 Is the design correct for the whole network?

Yes. Each journey is O(1) in time and each card is 8 bytes (24 bytes with a pass in Task 12), so 3,000,000 journeys a day are processed in linear time. The daily network total of about 5.9 x 10^9 paise is far below LLONG_MAX (9.2 x 10^18), and integer paise give the exact total at every scale tested (20,000, 500,000 and 3,000,000 journeys).

11.5 Alternative concept and complexity analysis

Task11/task11_config.cpp reads the fare table from fares.cfg when the gate starts. Reading z fare values takes O(z) time and O(z) space; it was measured at about 10 to 45 microseconds once per start-up, and the table occupies 48 bytes. This is negligible compared with recompiling and redeploying the program after every revision. The risk is a wrong or missing file, so the loader validates every value (present, non-negative, increasing by zone) and falls back to the built-in constexpr table:

```
== fares.cfg : loaded  [45.5 microseconds]
   Inner 10.00  Middle 20.00  Outer 30.00  Outer peak 36.00  min balance 50.00
== fares_revised.cfg : loaded  [10.5 microseconds]
   Inner 10.00  Middle 20.00  Outer 35.00  Outer peak 42.00  min balance 50.00
== fares_bad.cfg : REJECTED (negative value for MIDDLE), using built-in defaults  [5.4 microseconds]
   Inner 10.00  Middle 20.00  Outer 30.00  Outer peak 36.00  min balance 50.00
== missing.cfg : REJECTED (cannot open file), using built-in defaults  [1.5 microseconds]
   Inner 10.00  Middle 20.00  Outer 30.00  Outer peak 36.00  min balance 50.00
Space for the table: 48 bytes
```

Recommendation: keep the constexpr table as the verified default and allow a validated configuration file for revisions, so that fare changes do not require a rebuild.

TASK 12: DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

Changed requirement: a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

A. Impact analysis

- **enum values:** Zone needs a new value Airport; a new enumeration DayType (Weekday, Weekend) is needed.
- **Constants:** BASE_PAISE needs a fourth value 6000; a new WEEKEND_DISCOUNT_PERCENT = 10 is needed.
- **Expressions:** the surcharge expression must become a three-way time rule: weekend discount, else peak surcharge, else nothing.
- **switch statements:** every switch on Zone needs a case for Airport; the fare(Zone, bool peak) signature cannot express the weekend.
- **Classes:** Journey needs the day type, the month and the origin and destination zones; SmartCard needs an optional MonthlyPass; a Gate is needed to decide between pass and fare. The Rs 50 rule in SmartCard is unaffected.

B. Evaluation of the original design

Task12/task12_original_extended.cpp adds only Airport to the enum, as a developer might do in a hurry. The compiler warns and the program charges nothing for the Airport zone:

```
$ g++ -std=c++17 -Wall -Wextra  -o task12_original_extended task12_original_extended.cpp
task12_original_extended.cpp: In function 'long long int fare(Zone, bool)':
task12_original_extended.cpp:13:12: warning: enumeration value 'Airport' not handled in switch [-Wswitch]
   13 |     switch (z) {                                                // -Wswitch: 'Airport' not handled
      |            ^
$ ./task12_original_extended
Airport off-peak fare from the old switch: 0 paise (should be 6000)
Outer peak (weekday) still correct: 3600 paise
[exit code 0]
```

- **Worked example 1:** Airport off-peak: the switch has no Airport case, base stays 0, fare = Rs 0 instead of Rs 60.
- **Worked example 2:** Airport on a Saturday at 09:00: fare(z, peak) would add the 20% surcharge (Rs 72), but the requirement is the weekend discount only (60 x 0.9 = Rs 54). The bool parameter cannot express that the two rules do not combine.
- **Worked example 3:** Inner to Middle with a pass for Inner and Middle: the original design has no concept of a pass and charges Rs 20.

Conclusion: the SmartCard class absorbs the changes without modification (its rule is independent of how the fare is computed), but the switch-based fare logic cannot absorb them.

C. Adaptation: MODIFY

Decision: MODIFY. SmartCard is retained, the fare logic is modified to a table indexed by zone with a separate time rule, and MonthlyPass and Gate are added (Task12/task12_metro_v2.cpp).

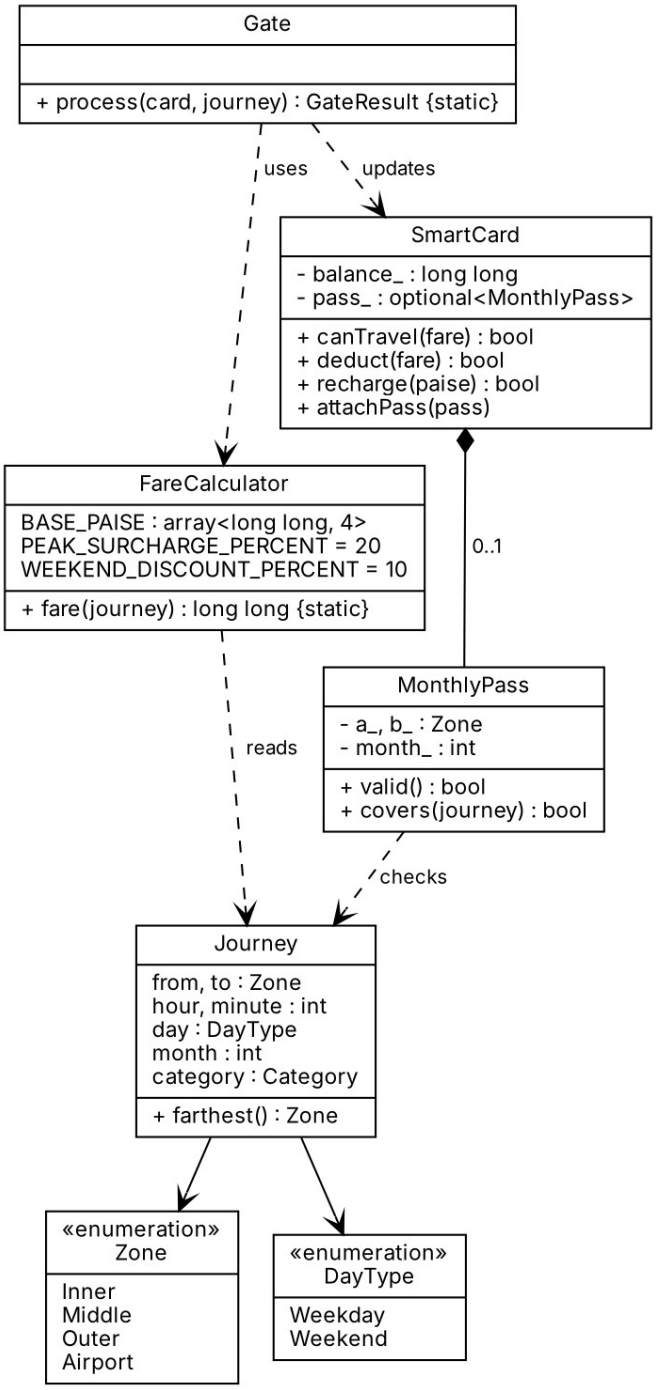


Figure 2: UML class diagram of the revised design

```
constexpr std::array<long long, ZONE_COUNT> BASE_PAISE = {1000, 2000, 3000, 6000};
static_assert(BASE_PAISE.size() == ZONE_COUNT, "every zone needs a fare");

static long long fare(const Journey& j) {
    long long f = BASE_PAISE[static_cast<std::size_t>(j.farthest())];
    if (j.day == DayType::Weekend)        f = applyPercent(f, 100 - WEEKEND_DISCOUNT_PERCENT);
    else if (isPeak(j.hour, j.minute))    f = applyPercent(f, 100 + PEAK_SURCHARGE_PERCENT);
```

```
    if (j.category == Category::Student)  f = applyPercent(f, 100 - STUDENT_CONCESSION_PERCENT);
    else if (j.category == Category::Senior) f = applyPercent(f, 100 - SENIOR_CONCESSION_PERCENT);
    return f;
}

bool MonthlyPass::covers(const Journey& j) const {
    return valid() && j.month == month_ && in(j.from) && in(j.to);   // both zones inside the pass
}

GateResult Gate::process(SmartCard& card, const Journey& j) {
    if (card.pass() && card.pass()->covers(j)) return {true, true, 0};
    const long long f = FareCalculator::fare(j);
    if (!card.deduct(f)) return {false, false, 0};
    return {true, false, f};
}
```

Assumptions: the weekend discount applies all day and replaces the peak surcharge; the concession is applied after it; a pass covers a journey only if both its origin and destination zones are in the pass and the journey is in the pass month.

D. Evidence-based defence

```
ID   Test case                                          Expected  Actual    Result
-------------------------------------------------------------------------------
R1   Inner weekday off-peak                             10.00     10.00     PASS
R2   Middle weekday peak 18:15                          24.00     24.00     PASS
R3   Outer weekday peak 08:00                           36.00     36.00     PASS
R4   Outer weekday 10:00 (off-peak)                     30.00     30.00     PASS
R5   Outer weekday peak senior                          21.60     21.60     PASS
A1   Airport weekday off-peak                           60.00     60.00     PASS
A2   Airport weekday peak 17:00                         72.00     72.00     PASS
A3   Airport weekday peak student                       54.00     54.00     PASS
D1   Outer weekend off-peak (30 x 0.9)                  27.00     27.00     PASS
D2   Airport weekend at 09:00 peak hour (60 x 0.9)      54.00     54.00     PASS
D3   Outer weekend student (30 x 0.9 x 0.75)            20.25     20.25     PASS
D4   Airport weekend senior (60 x 0.9 x 0.6)            32.40     32.40     PASS
P1   Pass: Inner->Middle in October is free             0.00      0.00      PASS
P2   Pass: balance unchanged after P1                   200.00    200.00    PASS
P3   Pass: Middle->Outer is outside the pass            30.00     30.00     PASS
P4   Pass: November journey, pass expired               10.00     10.00     PASS
P5   Pass with two identical zones is invalid           invalid   invalid   PASS
M1   Rs 110 - Airport Rs 60 leaves Rs 50: allowed       yes       yes       PASS
M2   Rs 109.99 - Airport Rs 60: denied                  no        no        PASS
-------------------------------------------------------------------------------
Passed 19, failed 0
Fare table: 32 bytes for 4 zones; sizeof(SmartCard) = 24 bytes
```

- **Regression:** R1 to R5 show that every weekday fare from Task 9 is unchanged, so the modification did not break the existing behaviour.
- **New zone:** A1 to A3 give Rs 60.00, Rs 72.00 and Rs 54.00 instead of Rs 0 from the original switch.

- **Weekend rule:** D2 gives Rs 54.00 for Airport at 09:00 on a weekend, proving that the discount and the surcharge do not combine; D3 and D4 show exact paise with concessions (Rs 20.25, Rs 32.40).
- **Pass:** P1 to P5 show a free journey inside the pass, a charged journey outside it, an expired pass and an invalid pass.
- **Minimum balance:** M1 and M2 show that the Rs 50 rule still holds exactly at the boundary for the new Rs 60 fare.

Trade-offs: the fare is still O(1) per journey; the fare table grows from 24 to 32 bytes and SmartCard from 8 to 24 bytes because of the optional pass, which is negligible for millions of cards. A fifth zone now needs only one value in the array (the static_assert stops a missing fare at compile time), while the original design needed a change in every switch and failed silently with fare 0. Retaining SmartCard kept the most important rule unchanged and tested. Therefore MODIFY is preferred over RETAIN (which fails A1 to D4) and over a full REDESIGN (which would discard the tested SmartCard and FareCalculator logic without any benefit).

CONCLUSION

The fare system was analysed as a set of objects, implemented with typed constexpr constants, an enum class, integer paise and a SmartCard class that protects its balance, and validated with 30 hand-written checks, 17 table-driven cases and 19 tests for the changed requirements, all of which pass. The measurements show that integer paise in long long are the only representation that is exact at the scale of the whole network, and the modified design absorbs the Airport zone, the weekend discount and the monthly pass while keeping every original fare unchanged.